



UNINASSAU

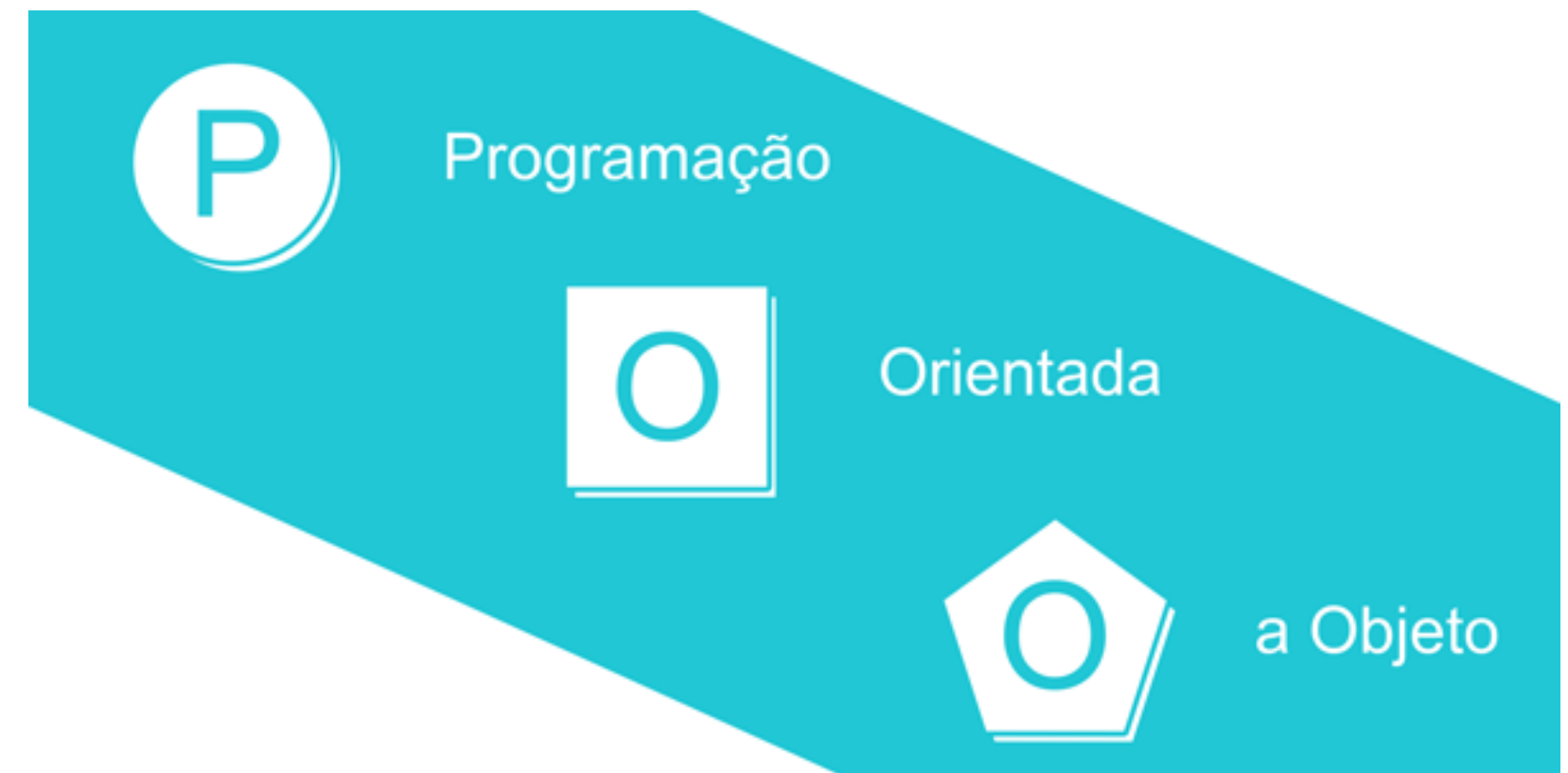
Instanciação e Referência de Objetos / Envio de Mensagens em Java

Análise e Desenvolvimento de Sistemas
Programação Orientada à Objetos e Estruturas de dados
Profº Eng. Esp. Lindenberg Andrade

Conceitos Fundamentais

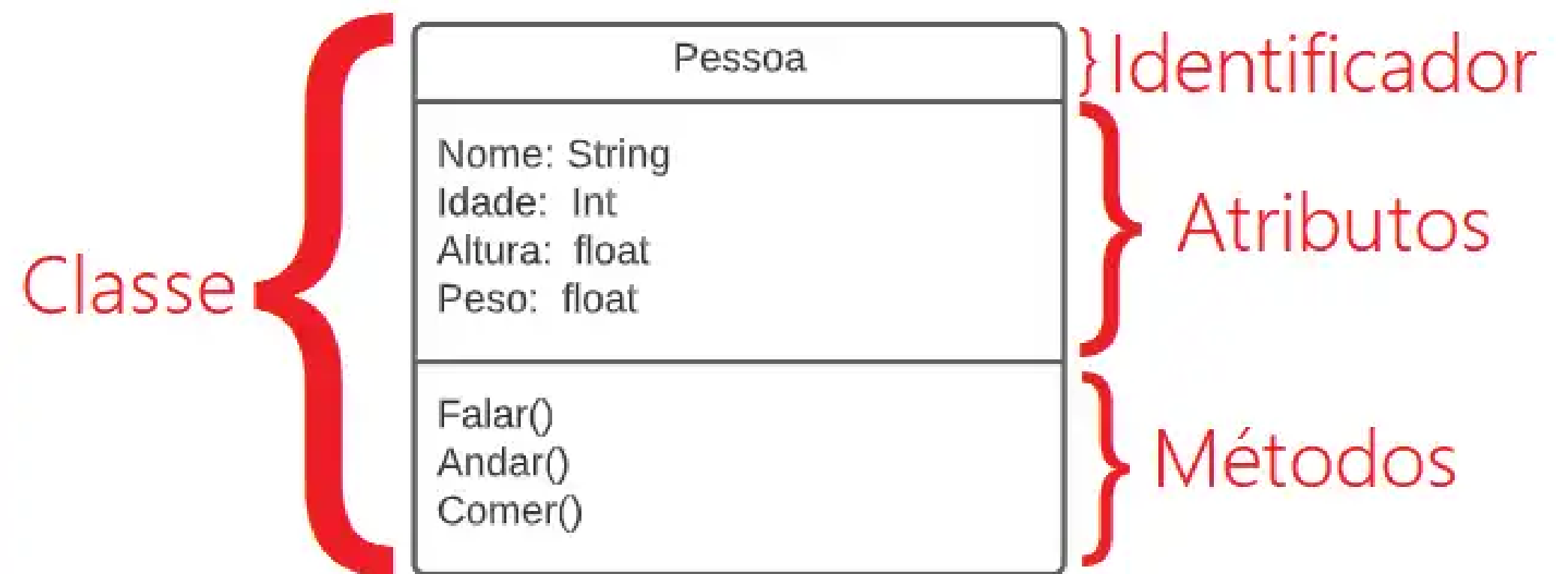
OBJETIVOS DA AULA

- Compreender criação de objetos em Java
- Diferenciar instância e referência
- Entender envio de mensagens (chamada de métodos)
- Aplicar conceitos em exemplos práticos



REVISÃO RÁPIDA POO

- Classe: Modelo ou molde do objeto
- Objeto: Instância da classe
- Atributos: Características ou propriedades do objeto
- Métodos: Comportamentos ou funções do objeto



INSTANCIACÃO DE OBJETOS

- Criar um objeto na memória usando

```
Pessoa p = new Pessoa();
```


REFERÊNCIA DE OBJETO

- Variável que aponta para o objeto na memória
- Exemplo:

```
Pessoa p1 = new Pessoa();
```

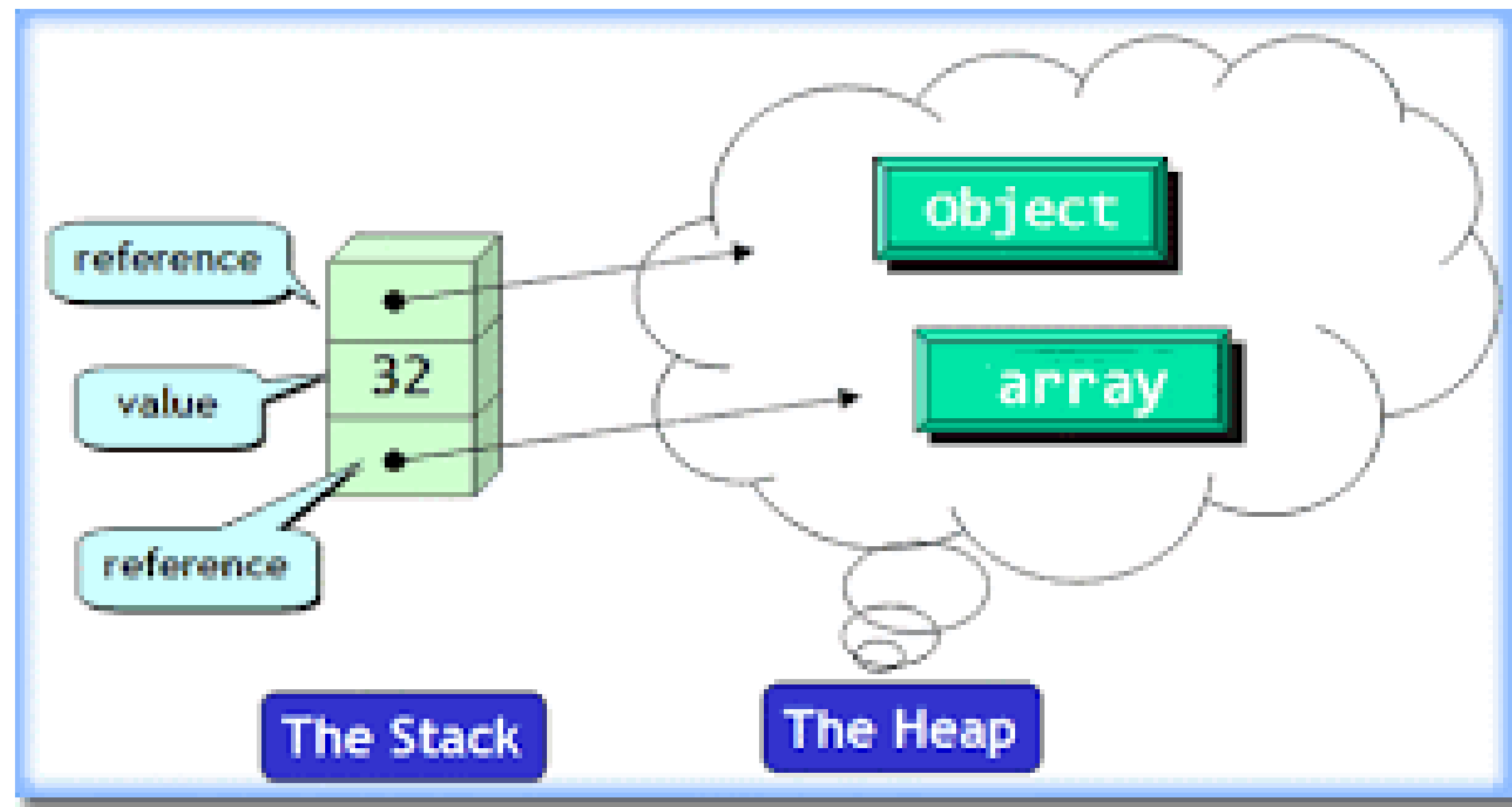
```
Pessoa p2 = p1; // p2 referencia o mesmo objeto que p1
```

DIFERENÇA: OBJETO X REFERÊNCIA

Conceito	Descrição	Exemplo
Objeto	Área de memória com dados	<code>new Pessoa()</code>
Referência	Variável que aponta para objeto	<code>Pessoa p1 = ...</code>

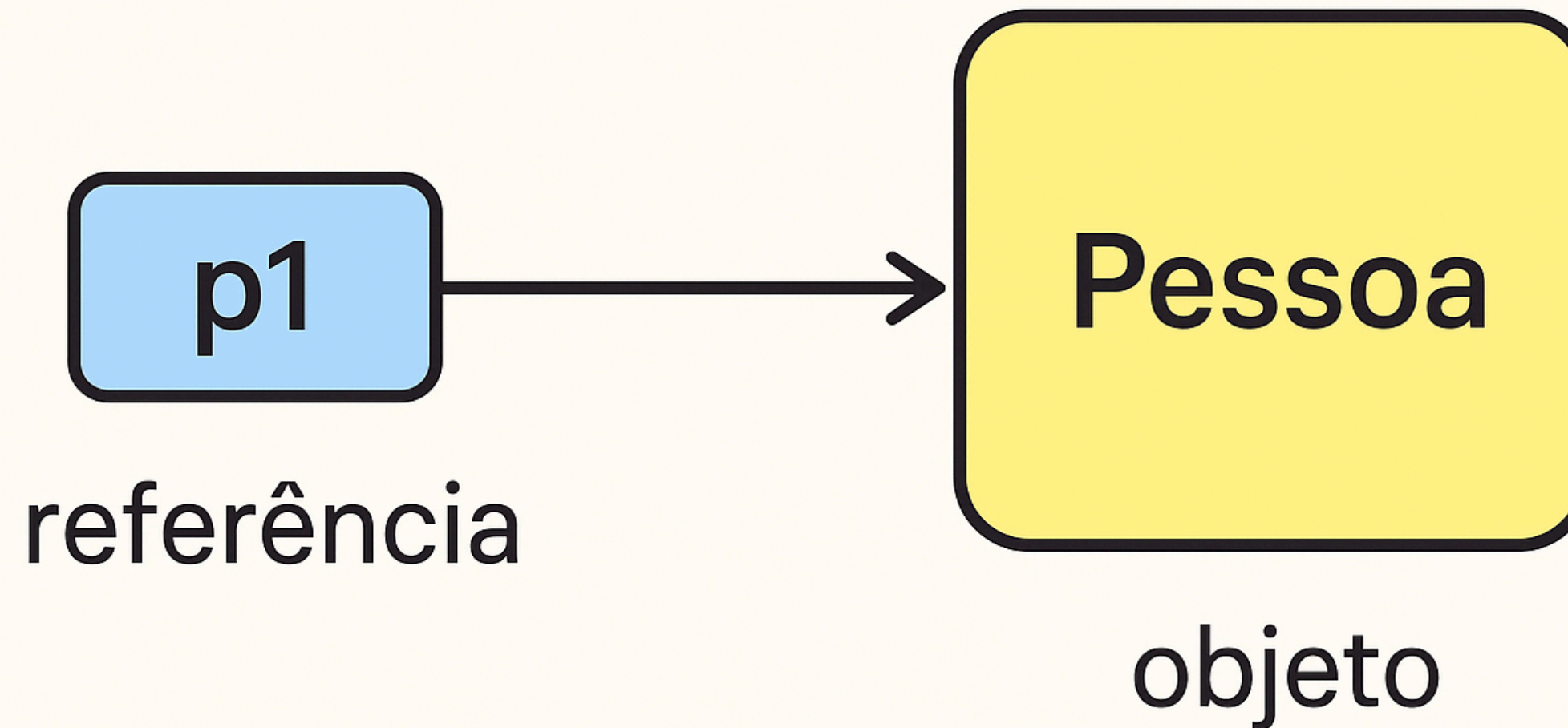
MEMÓRIA EM JAVA

- **Stack:** guarda referências
- **Heap:** guarda objetos reais
- **Diagrama visual:** referência → objeto na heap



EXEMPLO VISUAL

- Código: **Pessoa p1 = new Pessoa();**
- Exemplo: Caixa **p1** apontando para objeto **Pessoa**



CONSTRUTOR PADRÃO E PARAMETRIZADO

- **Padrão: sem parâmetros**

```
Carro c = new Carro();
```

- **Parametrizado: inicializa atributos**

```
Carro c = new Carro("Fusca", 1970);
```

Trabalhando com Referências

ATRIBUIÇÃO DE REFERÊNCIA

- Múltiplas referências podem apontar para o mesmo objeto

```
Pessoa p1 = new Pessoa();  
Pessoa p2 = p1;
```

ALTERAÇÃO VIA REFERÊNCIA

- Alterar objeto via qualquer referência altera todos que apontam para ele

```
p1.setNome("João");
```

```
System.out.println(p2.getNome()); // João
```

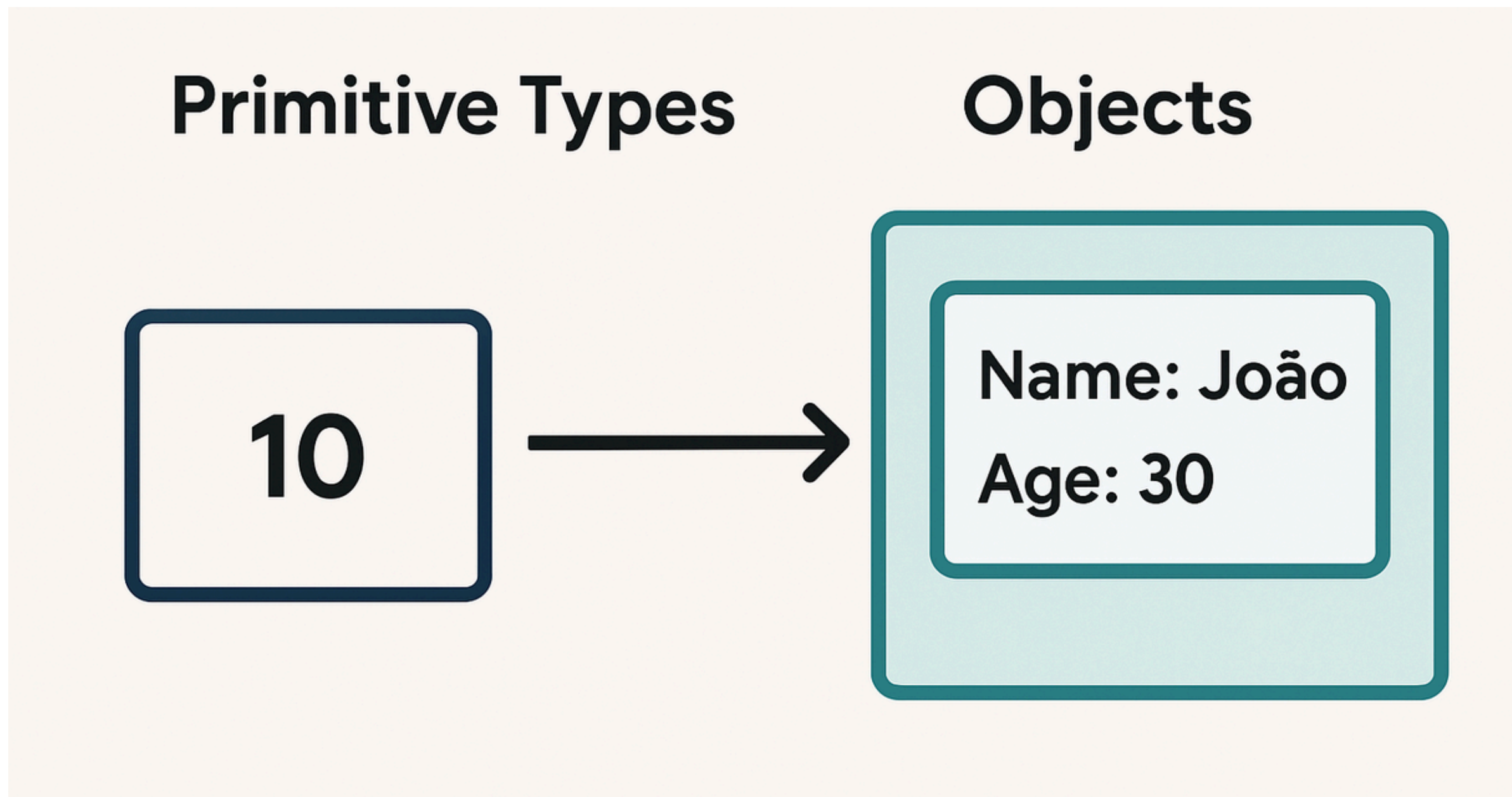

COMPARANDO OBJETOS

== → compara referências

equals() → compara conteúdo

TIPOS PRIMITIVOS VS OBJETOS

- Tipos primitivos armazenam valor direto
- Objetos armazenam endereço (referência)



NULL E REFERÊNCIAS

- Referência nula (**null**) não aponta para objeto

```
Pessoa p = null;
```

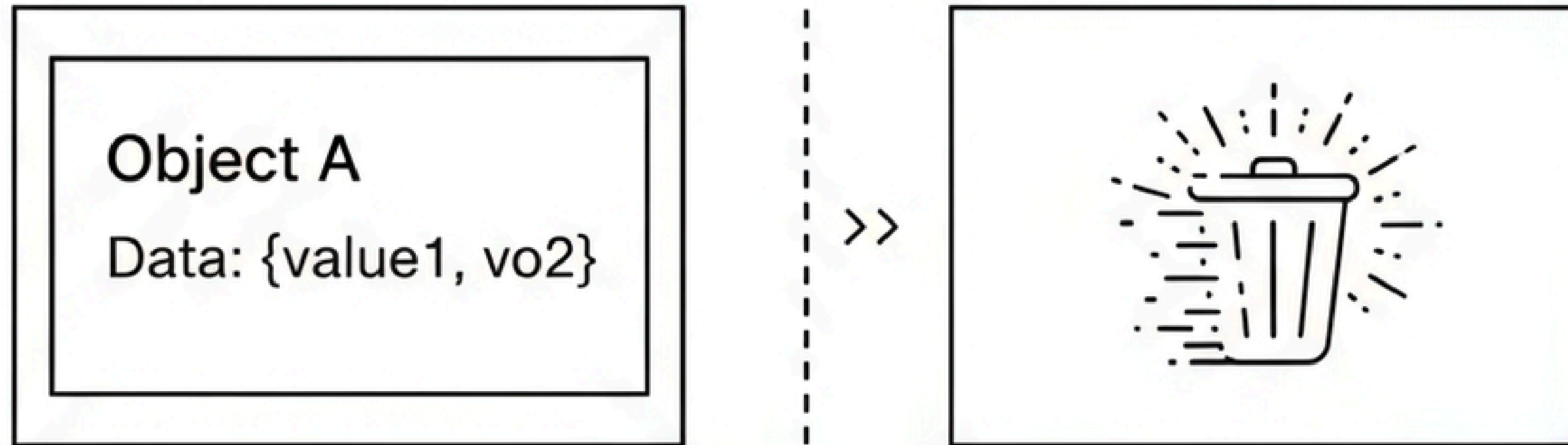

EVITANDO NULLPOINTEREXCEPTION

- Verificar se referência não é nula antes de usar

```
if(p != null) p.mostrarNome();
```

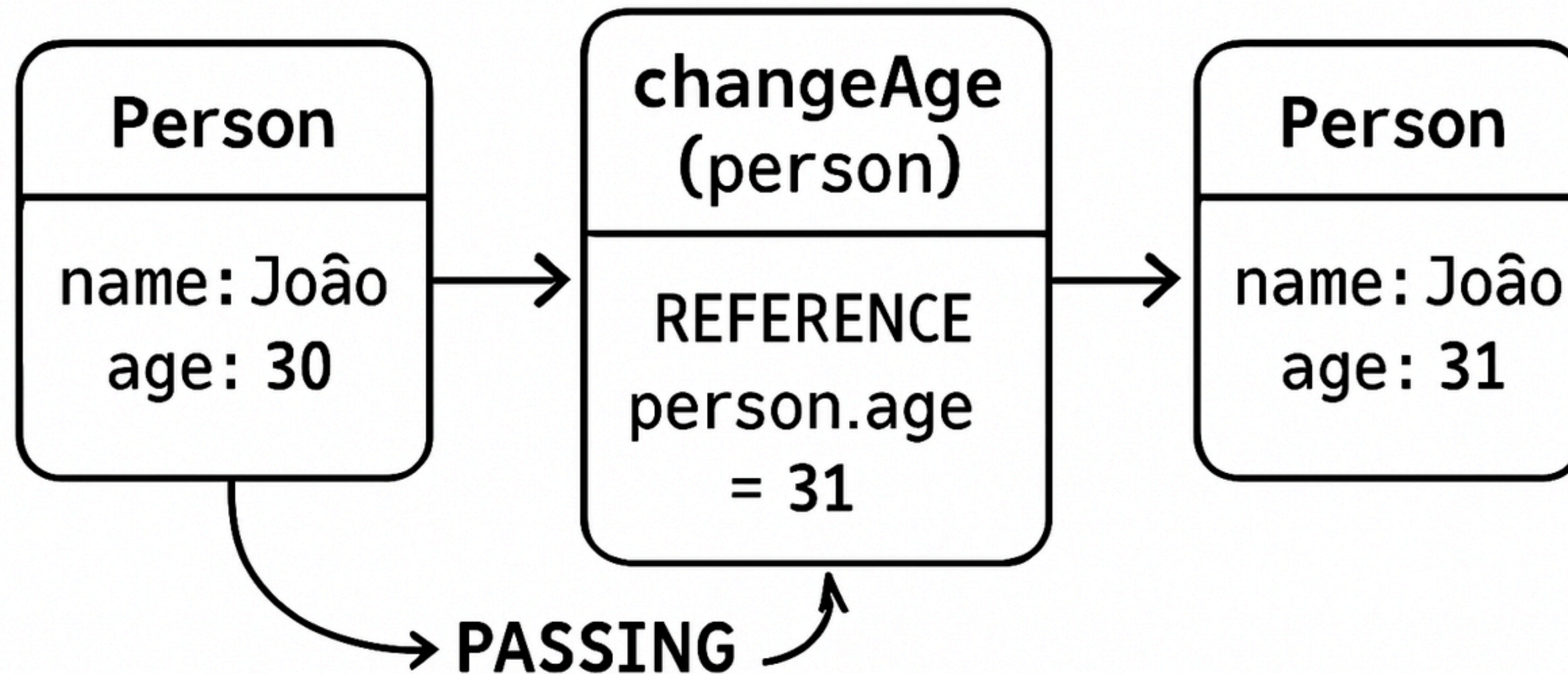
GARBAGE COLLECTOR

- Objetos sem referência são coletados automaticamente
- Ex: Diagrama mostra objeto sem referência sendo removido



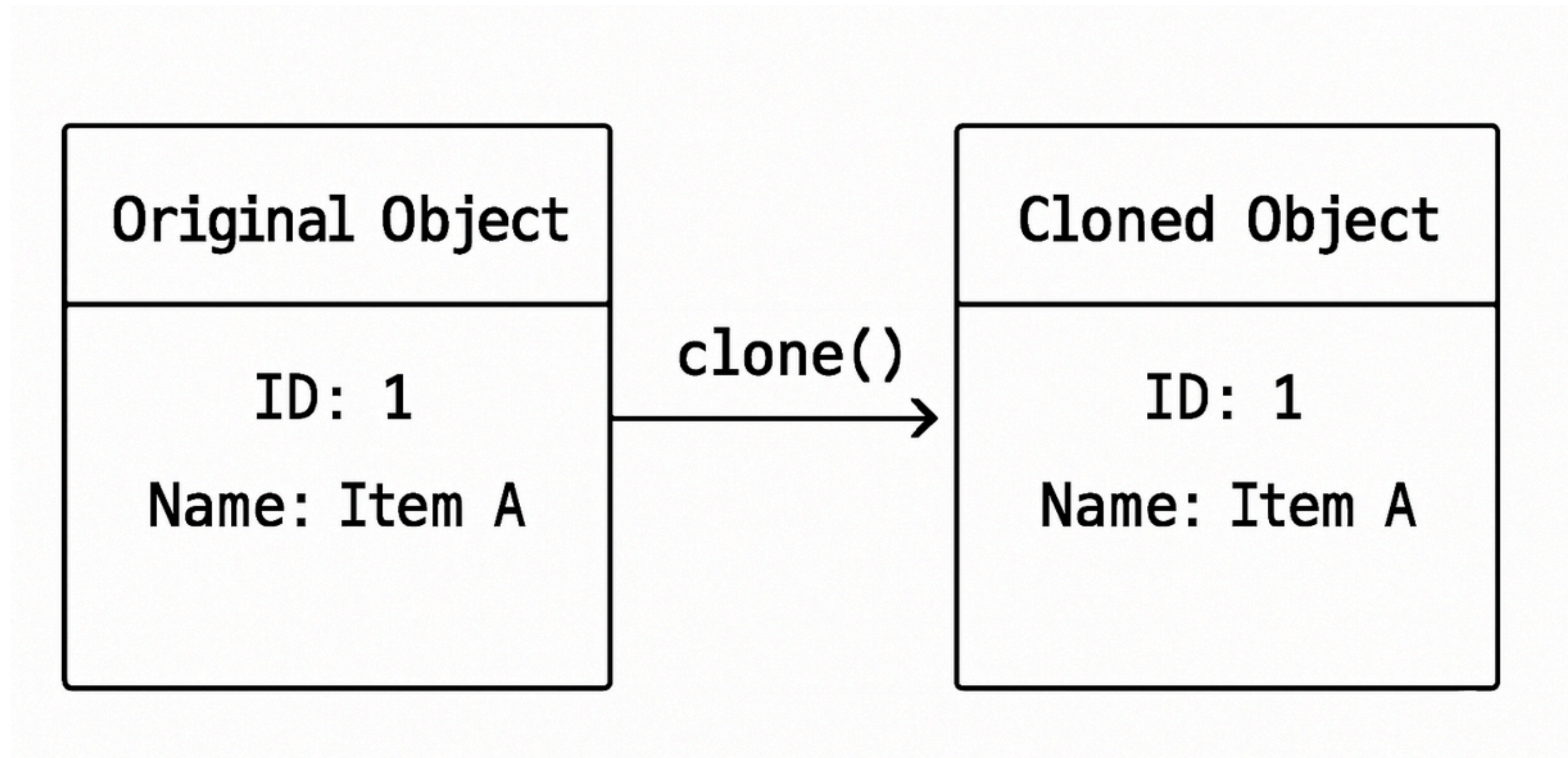
PASSAGEM DE OBJETOS PARA MÉTODOS

- Referência é passada por valor
- Exemplo: alteração de atributo dentro do método afeta o objeto



CLONAGEM DE OBJETOS

- **clone()** ou construtor de cópia
- Evita efeitos de múltiplas referências



ARRAYS DE OBJETOS

```
Pessoa[] pessoas = new Pessoa[3];  
pessoas[0] = new Pessoa("Ana");
```

LISTAS DE OBJETOS

```
ArrayList<Pessoa> lista = new ArrayList<>();  
lista.add(new Pessoa("Carlos"));
```

Envio de Mensagens

CONCEITO DE ENVIO DE MENSAGENS

- Na POO, objetos interagem entre si **enviando mensagens**.
- Uma **mensagem** é uma chamada de método de um objeto para outro.
- Exemplo conceitual:
 1. Objeto **Aluno** quer saber a nota de um **Professor**.
 2. **Aluno** envia uma mensagem (chamada de método) ao **Professor** para obter a nota.
- **Analogia do mundo real:**

"Enviar mensagem" é como pedir a alguém para fazer algo: você não faz o trabalho, apenas solicita.

COMO FUNCIONA EM JAVA

- Cada objeto possui **métodos**.
- Enviar uma mensagem = **chamar um método de outro objeto**.
- Sintaxe básica:

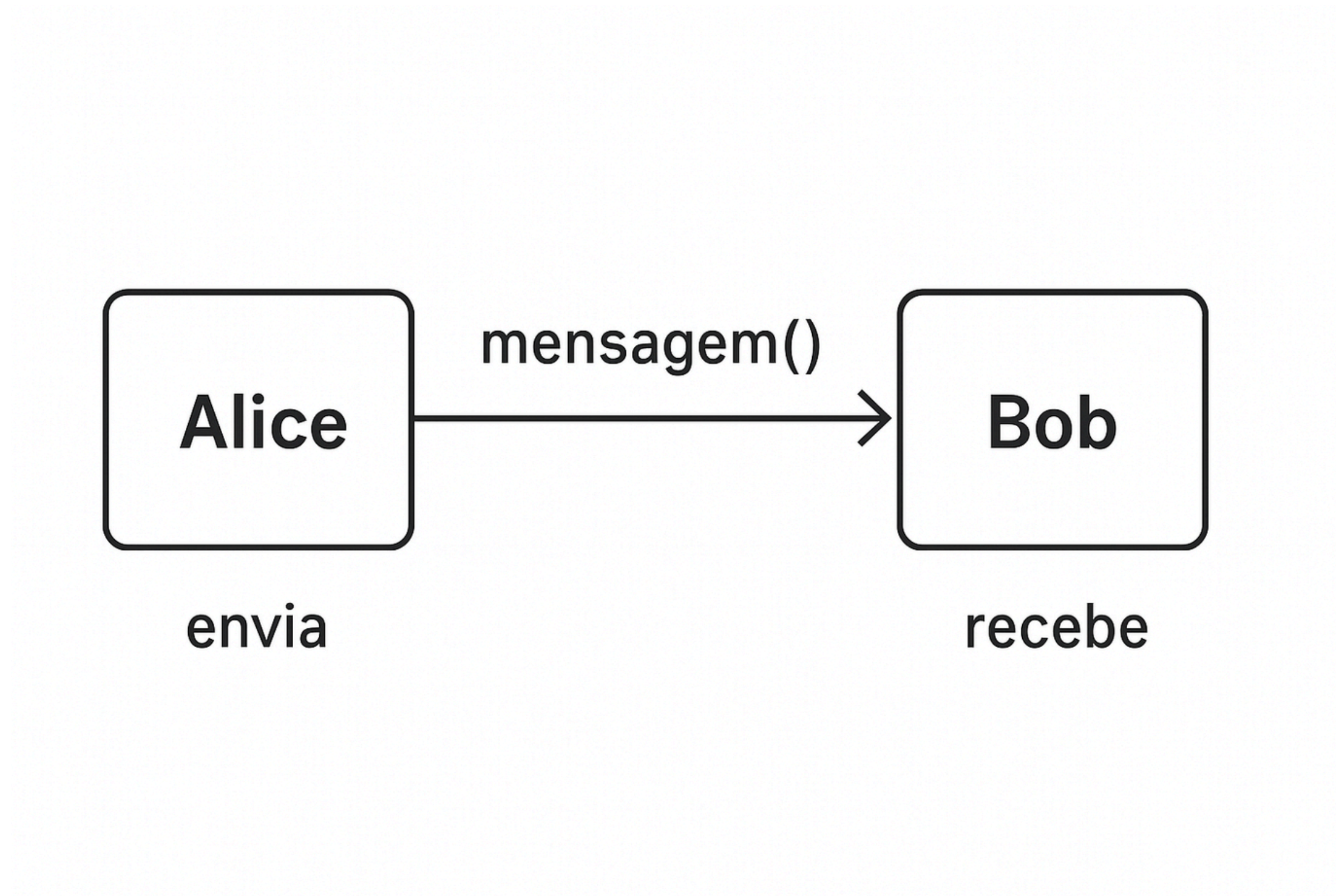
objeto.mensagem(parametros);

objeto → quem recebe a mensagem.

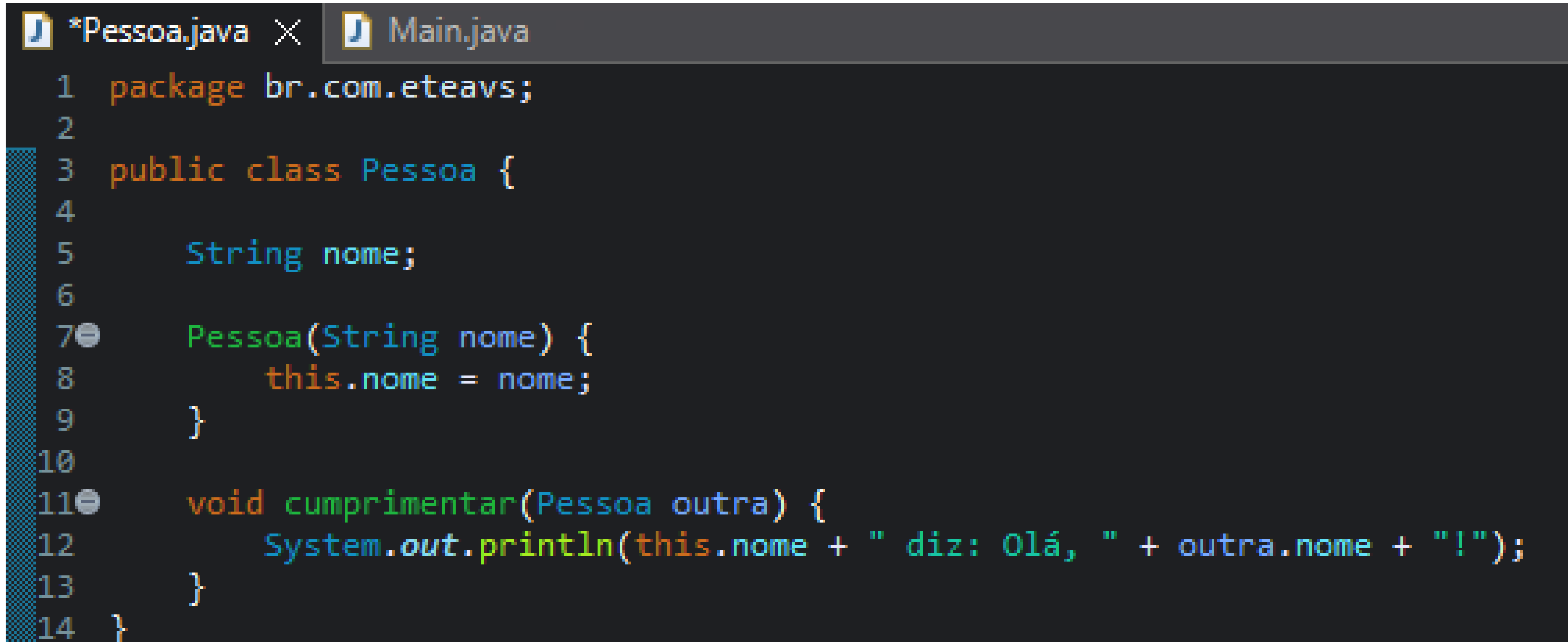
mensagem → o método chamado.

parametros → informações enviadas junto com a mensagem.

EXEMPLO VISUAL

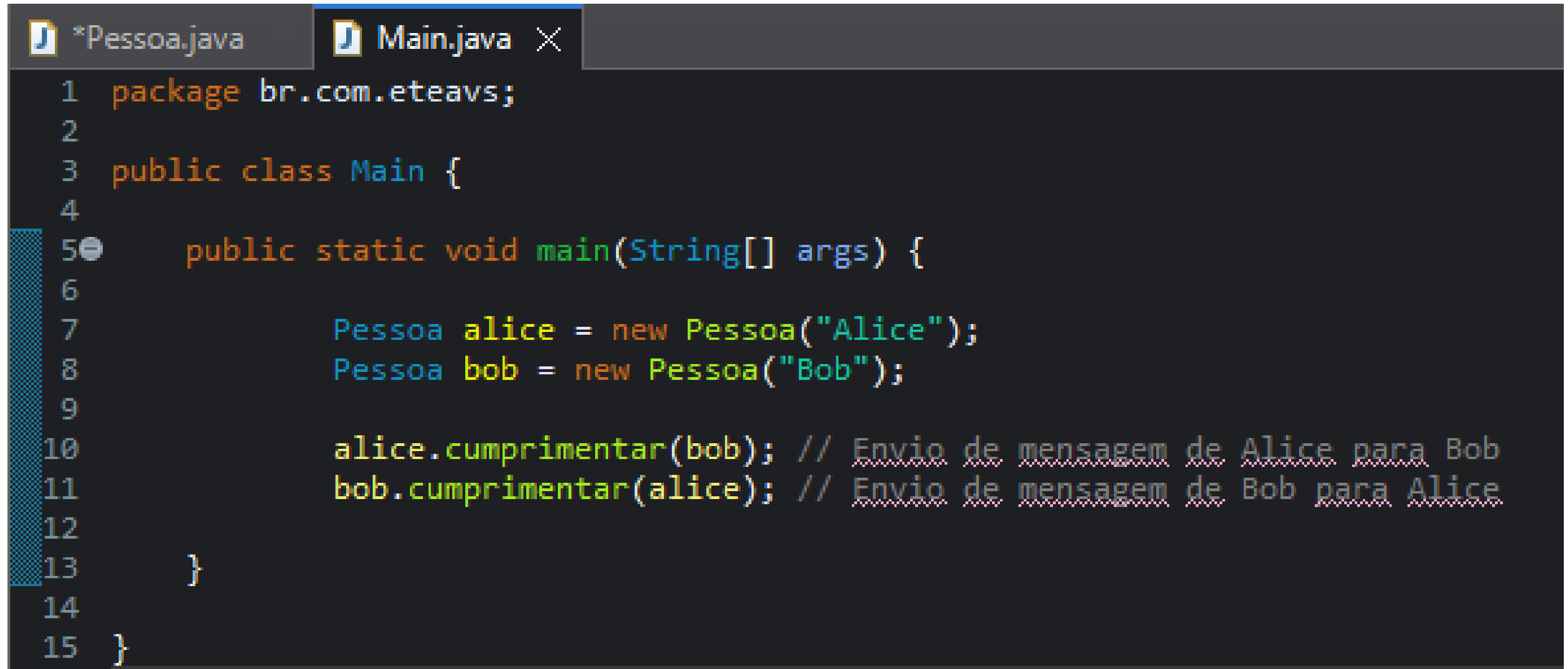


EXEMPLO PRÁTICO 1: COMUNICAÇÃO ENTRE OBJETOS



```
*Pessoa.java × Main.java
1 package br.com.eteavs;
2
3 public class Pessoa {
4
5     String nome;
6
7     Pessoa(String nome) {
8         this.nome = nome;
9     }
10
11     void cumprimentar(Pessoa outra) {
12         System.out.println(this.nome + " diz: Olá, " + outra.nome + "!");
13     }
14 }
```

EXEMPLO PRÁTICO 1: COMUNICAÇÃO ENTRE OBJETOS

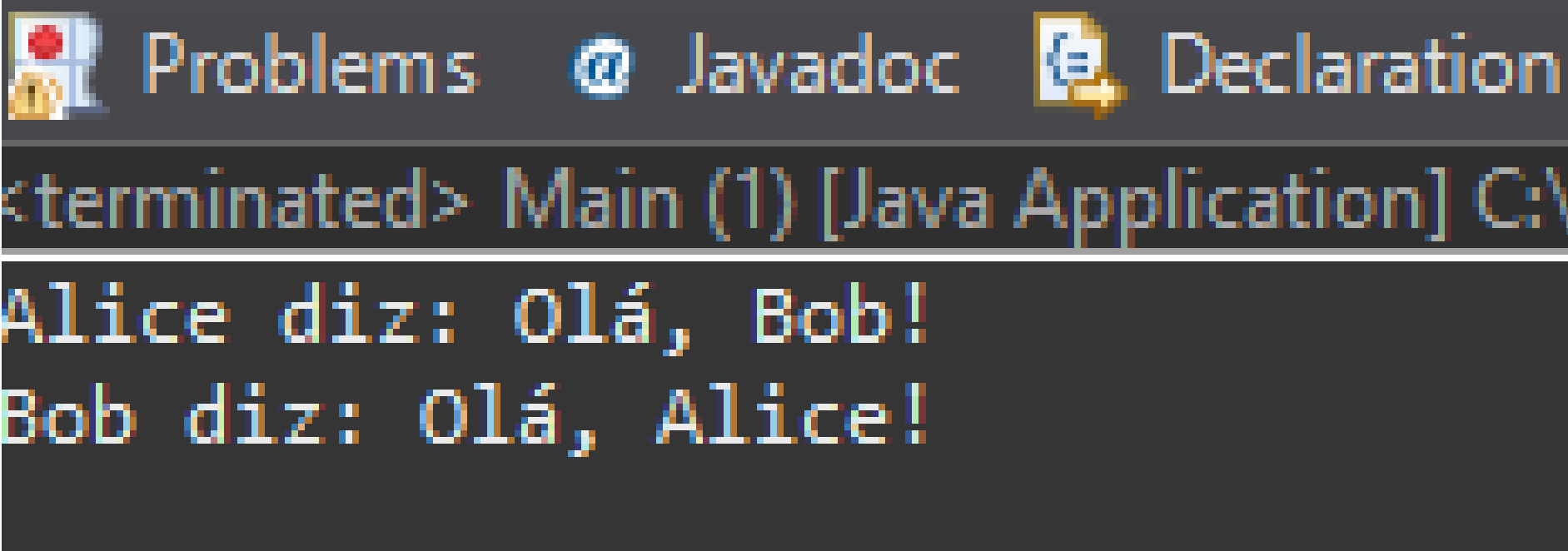


```
1 package br.com.eteavs;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         Pessoa alice = new Pessoa("Alice");
8         Pessoa bob = new Pessoa("Bob");
9
10        alice.cumprimentar(bob); // Envio de mensagem de Alice para Bob
11        bob.cumprimentar(alice); // Envio de mensagem de Bob para Alice
12
13    }
14
15 }
```

Observação: **alice.cumprimentar(bob)** é o envio de mensagem de Alice para Bob.

EXEMPLO PRÁTICO 1: COMUNICAÇÃO ENTRE OBJETOS

Saída Esperada:

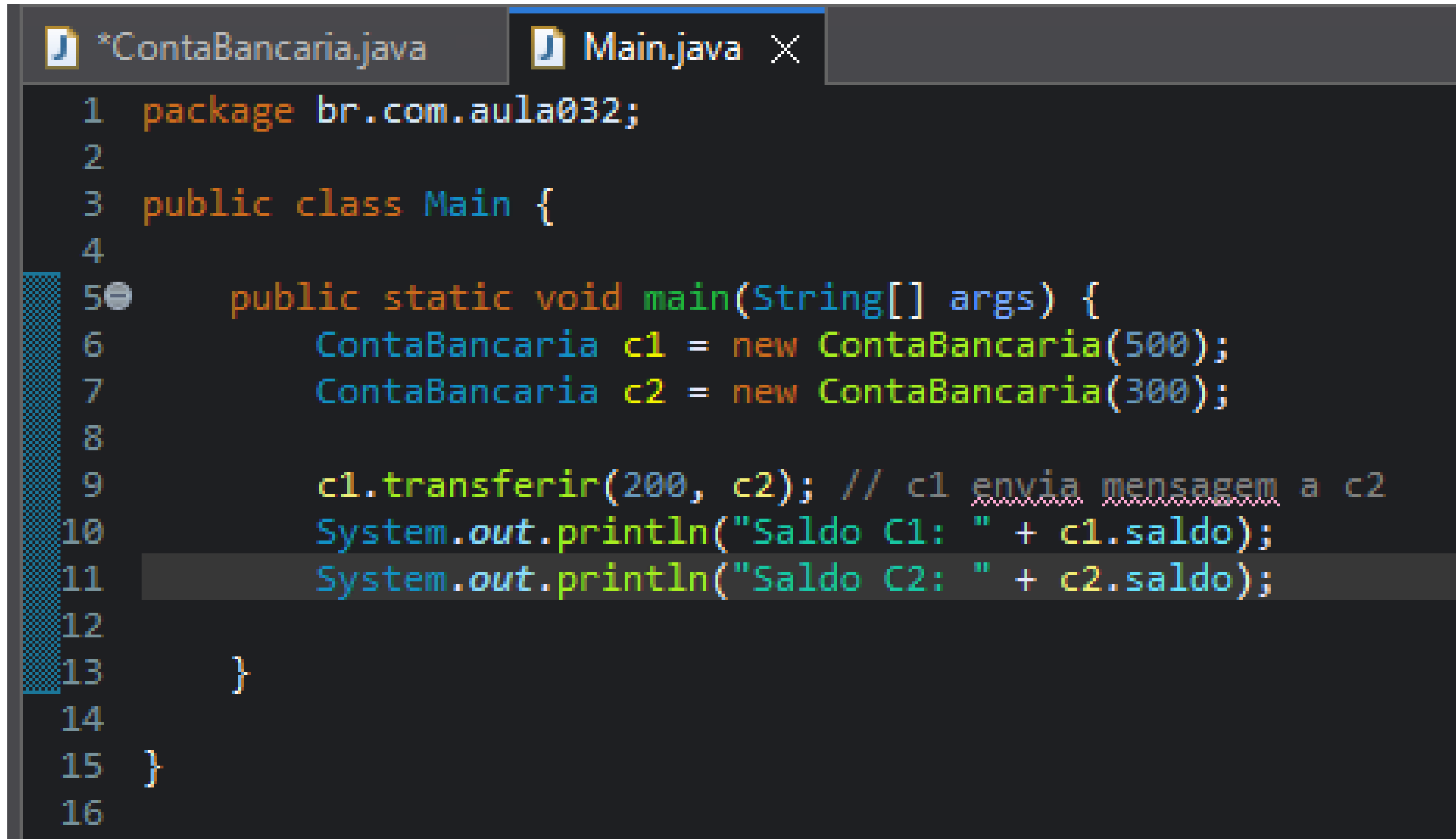


```
Problems  @ Javadoc  Declaration  
<terminated> Main (1) [Java Application] C:\  
Alice diz: Olá, Bob!  
Bob diz: Olá, Alice!
```


EXEMPLO PRÁTICO 2: ENVIO DE DADOS VIA MENSAGENS

```
*ContaBancaria.java × Main.java
1 package br.com.aula032;
2
3 public class ContaBancaria {
4     double saldo;
5
6     ContaBancaria(double saldoInicial) {
7         saldo = saldoInicial;
8     }
9
10    void transferir(double valor, ContaBancaria destino) {
11        if(saldo >= valor) {
12            saldo -= valor;
13            destino.saldo += valor;
14            System.out.println("Transferência de R$" + valor + " realizada.");
15        } else {
16            System.out.println("Saldo insuficiente!");
17        }
18    }
19 }
```

EXEMPLO PRÁTICO 2: ENVIO DE DADOS VIA MENSAGENS

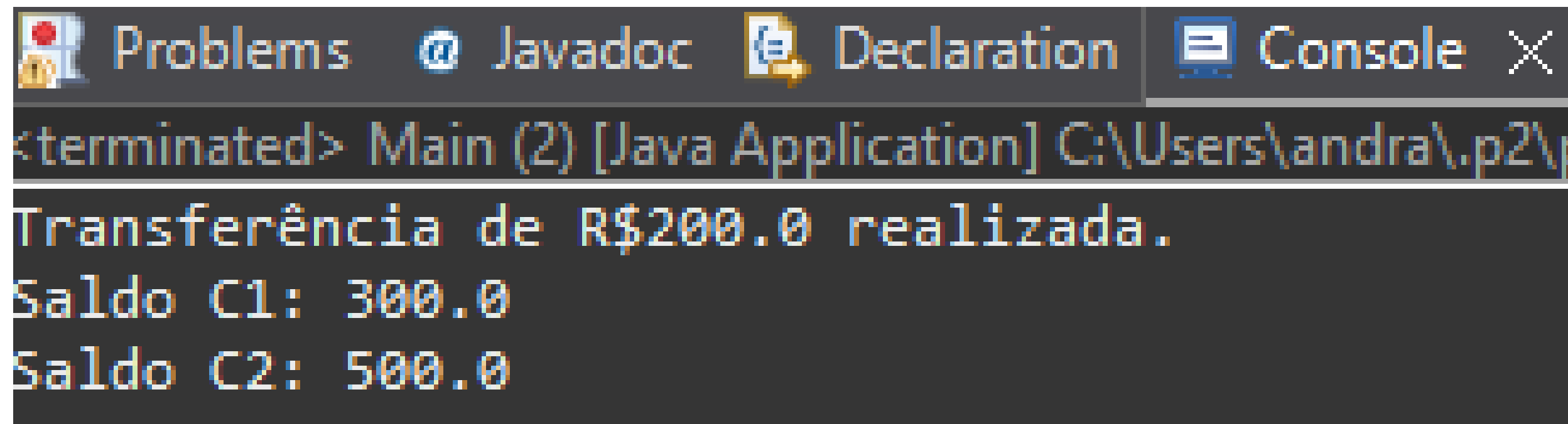


```
1 package br.com.aula032;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         ContaBancaria c1 = new ContaBancaria(500);
7         ContaBancaria c2 = new ContaBancaria(300);
8
9         c1.transferir(200, c2); // c1 envia mensagem a c2
10        System.out.println("Saldo C1: " + c1.saldo);
11        System.out.println("Saldo C2: " + c2.saldo);
12
13    }
14
15 }
16
```

Observação: o envio de mensagem permite que objetos interajam e modifiquem seu estado.

EXEMPLO PRÁTICO 2: ENVIO DE DADOS VIA MENSAGENS

Saída Esperada:



```
Problems  @ Javadoc  Declaration  Console  X
<terminated> Main (2) [Java Application] C:\Users\andra\.p2\p
Transferência de R$200.0 realizada.
Saldo C1: 300.0
Saldo C2: 500.0
```


REFERÊNCIAS

1. BARNES, David J.; KÖLLING, Michael. Programação orientada a objetos com Java: uma introdução prática usando o BlueJ. 4. ed. São Paulo: Pearson Prentice Hall, 2008.
2. SANTOS, Rafael. Introdução à Programação Orientada a Objetos usando Java. 2. ed. Rio de Janeiro: Elsevier, 2013. 336 p.
3. SERSON, Roberto Rubinstein. Programação orientada a objetos com Java 6 – curso universitário. 1. ed. Rio de Janeiro: Brasport, jun. 2009. 492 p.
4. PREISS, Bruno R. Estruturas de dados e algoritmos: padrões de projetos orientados a objetos com Java. Rio de Janeiro: Campus, 2001. 566 p.

The Electric State (2025)



The Electric State se passa numa realidade alternativa e retro futurista da década de 90 na qual robôs conscientes convivem entre os humanos. A história acompanha Michelle (Millie Bobby Brown), uma adolescente orfã que, um dia, recebe a visita de Cosmo, um robô meigo e misterioso que ela acredita estar sendo controlado por Christopher, seu irmão mais novo considerado morto. Determinada a saber a verdade sobre o irmão, Michelle parte numa aventura pelo oeste americano com a companhia indesejada e inesperada do contrabandista e andarilho Keats (Chris Pratt) e seu robô parceiro e sarcástico Herman (dublado por Anthony Mackie). Dirigido pelos Irmãos Russo (Vingadores: Ultimato), The Electric State é baseado na graphic novel de Simon Stålenhag.

Dúvidas?



Profº Lindenberg Andrade
E-mail: linndemberg1@gmail.com



Additional contacts via QR code